Tasca Sprint4
Modelat SQL

Peer: Erik Subirats. Erik me ha recomendado que en la tabla puente ponga primary keys en las dos columnas para evitar duplicados por errores, también ha visto un fallo en mi código, lo cual solucionamos juntos.

_ - _

Nivell 1

Descàrrega els arxius CSV, estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:

Yo creé una base de datos nueva, llamada global_transactions:

```
CREATE SCHEMA IF NOT EXISTS global_transactions;
```

Y a partir de ahora, usaremos nuestra nueva base de datos:

```
USE global_transactions;
```

Creamos y subimos los datos a las tablas:

```
CREATE TABLE transactions (
    id CHAR(36) PRIMARY KEY, INDEX(id),
    card_id VARCHAR(20),
    business_id VARCHAR(20),
    timestamp VARCHAR(50),
    amount DECIMAL(10,2),
    declined BOOLEAN,
    product_ids VARCHAR(255),
    user_id INT,
    lat DECIMAL(10,8),
    longitude DECIMAL(11,8)
);
```

Subimos los datos a la tabla:

```
LOAD DATA LOCAL INFILE 'C:/transactions.csv'
INTO TABLE transactions
FIELDS TERMINATED BY ';'
ENCLOSED BY '"'
LINES TERMINATED BY
IGNORE 1 ROWS
(
    id, card_id, business_id,
    timestamp, amount, declined,
    product_ids, user_id, lat, longitude
);
```

La sintaxis arriba es la recomendada en la documentación de MySQL en caso de querer subir .CSV como datos para las tablas.

> LOAD DATA can be used to read files obtained from external sources. For example, many programs can export data in comma-separated values (CSV) format, such that lines have fields separated by commas and enclosed within double quotation marks, with an initial line of column names. If the lines in such a file are terminated by carriage return/newline pairs, the statement shown here illustrates the field- and line-handling options you would use to load the file:
>
> ```
> LOAD DATA INFILE 'data.txt' INTO TABLE tbl_name
>  FIELDS TERMINATED BY ',' ENCLOSED BY '"'
>  LINES TERMINATED BY '\r\n'
>  IGNORE 1 LINES;
> ```

`FIELDS TERMINATED BY ';'` -- separador del CSV. A veces pueden ser comillas.

`ENCLOSED BY '"'` -- que las cadenas están encerradas por comillas dobles

`LINES TERMINATED BY '\n'` -- salto de línea

`IGNORE 1 ROWS` -- ignorar cabecera

Creo y alimento las otras tablas: **products, european_users, american_users, credit_cards, companies.** Uno las tablas american y european users en una única tabla, que se llamará users.

```sql
CREATE TABLE users AS
SELECT * FROM european_users
UNION ALL
SELECT * FROM american_users;
```

Aquí el `UNION ALL` funciona porque no hay conflicto con ID's. O bien podemos hacer un `IGNORE INDEX` si hubiera conflicto. Quedaría así:

```sql
CREATE TABLE users AS
SELECT * FROM european_users IGNORE INDEX (id)
UNION ALL
SELECT * FROM american_users IGNORE INDEX (id);
```

Un par de modificaciones para relacionar mejor las tablas

```sql
ALTER TABLE transactions
RENAME COLUMN business_id TO company_id;

ALTER TABLE transactions
RENAME COLUMN product_ids TO product_id;

ALTER TABLE users
ADD PRIMARY KEY (id);
```

Después de ver el reverse engineer, decidí quitar el plural de las tablas.

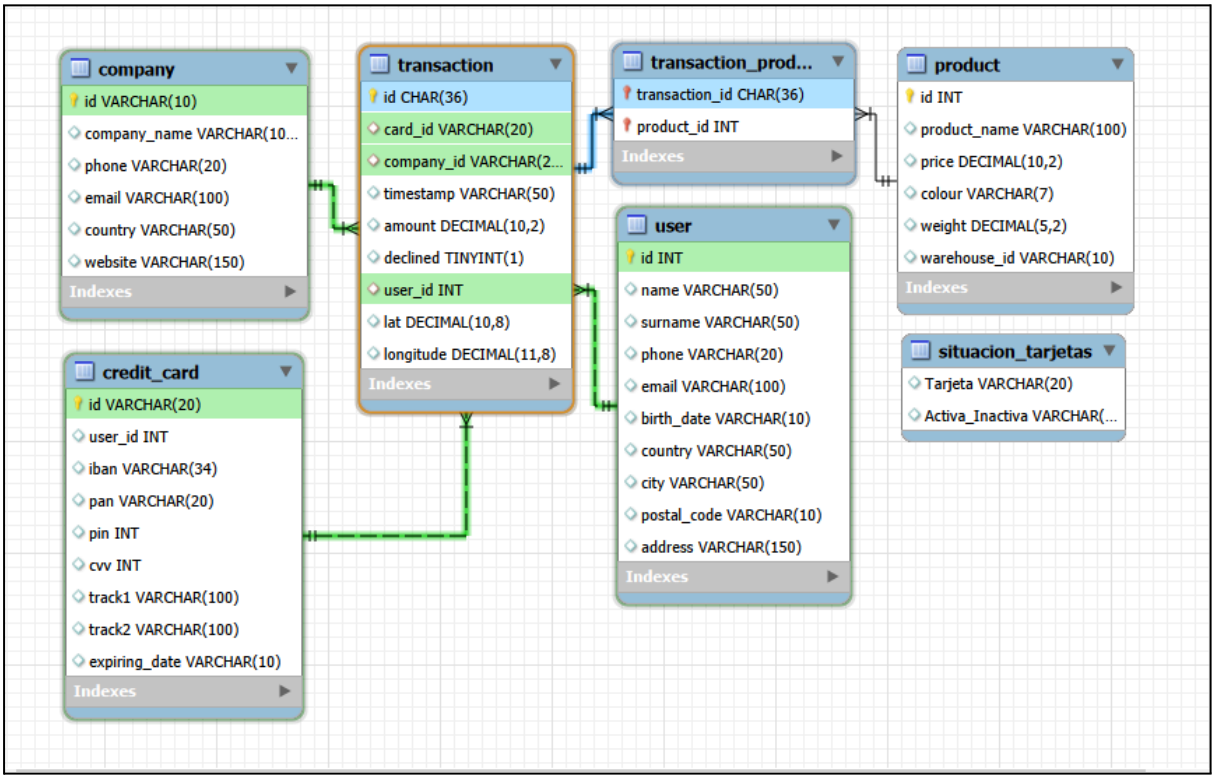
```sql
RENAME TABLE credit_cards TO credit_card;
RENAME TABLE products TO product
RENAME TABLE transactions TO transaction;
```

```sql
RENAME TABLE users TO user;

RENAME TABLE companies TO company;
```

Con todas las tablas creadas y modificadas, hago la última modificación, crear los enlaces entre ellas.

```sql
ALTER TABLE transaction

ADD CONSTRAINT fk_card

    FOREIGN KEY (card_id) REFERENCES credit_card(id),

ADD CONSTRAINT fk_users

    FOREIGN KEY (user_id) REFERENCES user(id),

ADD CONSTRAINT fk_companies

    FOREIGN KEY (company_id) REFERENCES company(id);
```

Ahora, con las relaciones y arreglos hechos, ese es nuestro database estrella. Aquí ya está incluida nuestra tabla puente del nivel 3.



Exericici 1

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

```
221   -- més de 80 transaccions utilitzant almenys 2 taules.
222   SELECT id AS Identificador,
223          CONCAT(name, ' ', surname) AS Nombre,
224          (SELECT COUNT(id)
225           FROM transaction
226           WHERE user_id = user.id) AS Transacciones
227   FROM user
228   WHERE id IN (SELECT transaction.user_id
229                FROM transaction
230                GROUP BY user_id
231                HAVING COUNT(id) > 80
232                )
233   ORDER BY Transacciones DESC
234   ;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Identificador	Nombre	Transacciones
185	Molly Gilliam	110
289	Dxwgi Hwcru	94
318	Bnyr Astuw	91
454	Sfzzoh Xgvfridxs	81

Result 7

Output

Action Output

#	Time	Action	Message
1	13:21:40	SELECT id AS Identificador, CONCAT(name, ' ', surname) AS Nombre, (SELECT CO...	4 row(s) returned

Concatenamos los nombres y hacemos una subconsulta en el `SELECT` para ver si se cumplen las condiciones.

Esa misma consulta podemos hacerla desde `transaction`, lo hacemos para comprobar resultados.

```
-- comprobamos con una query en transaction
SELECT user_id, COUNT(id)
FROM transaction
GROUP BY user_id
HAVING COUNT(id) > 80
ORDER BY COUNT(id) DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

user_id	COUNT(id)
185	110
289	94
318	91
454	81

Result 8

Output

Action Output

#	Time	Action	Message
1	13:24:16	SELECT user_id, COUNT(id) FROM transaction GROUP BY user_id HAVING COUNT(id) >...	4 row(s) returned

Exericici 2

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

```
-- utilitza almenys 2 taules.
SELECT iban AS Número_iban,
       round(AVG(amount),2) AS Media_Importe,
       c.company_name AS Compania
FROM transaction t
JOIN credit_card cc
ON t.card_id = cc.id
JOIN company c
ON t.company_id = c.id
WHERE c.company_name = 'Donec Ltd'
AND t.declined = 0
GROUP BY cc.iban, cc.id
ORDER BY Media_importe DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Número_iban	Media_Importe	Compania
XX3830178139196201993663 52	680.69	Donec Ltd
XX6377063573975703949739 13	680.01	Donec Ltd
XX9713939714652922023122 59	645.46	Donec Ltd
XX1718471169288923759693 07	628.89	Donec Ltd
XX2254246388185424062235 75	608.68	Donec Ltd
XX7488907290571957117660 71	607.29	Donec Ltd

Result 10

Output

Action Output

#	Time	Action	Message
1	13:28:14	SELECT iban AS Número_iban, round(AVG(amount),2) AS Media_Importe, c.compa...	370 row(s) returned

_ - _

Nivell 2

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:

```
CREATE TABLE situacion_tarjetas AS

SELECT card_id AS Tarjeta,

CASE

    WHEN SUM(declined) = 3 THEN 'Inactiva'

    WHEN MIN(declined) = 0 THEN 'Activa'

END AS Activa_Inactiva

FROM (SELECT card_id,

       declined,

       timestamp,

       ROW_NUMBER()

       OVER(PARTITION BY card_id ORDER BY timestamp DESC)

       AS últimos_registros

       FROM transaction) AS más_recientes

WHERE últimos_registros <= 3

GROUP BY card_id;
```

Ese fue el desafío del sprint porque me ha costado mucho entender algunos conceptos.

El `ROW_NUMBER()` Asigna un número de fila consecutivo (1, 2, 3...) a cada fila según el criterio que le indiques.

`PARTITION BY card_id` Le dice "reinicia el contador para cada tarjeta". Es decir, cada card_id tiene su propia numeración independiente:

card_id	timestamp	ROW_NUMBER
CcS-6894	2024-01-03	1
CcS-6894	2024-01-02	2
CcS-6894	2024-01-01	3
XYZ-1111	2024-01-03	**1 ← reinicia**
XYZ-1111	2024-01-02	2

Aquí no podemos hacer `LIMIT 3`; porque solamente me quedo con las 3 primeras filas y no con las 3 de cada tarjeta. Aquí utilizamos la condición `WHERE últimos_registros =< 3` porque itera sobre el número de registros de las tarjetas, en ese caso, los últimos 3 registros dado que tenemos un `ORDER BY timestamp` dentro del `PARTITION`.

Por último, utilizamos el `CASE WHEN` para definir las condiciones que queremos. Para la condición `Inactiva`, de que las tres últimas operaciones fueron rechazadas, utilicé un `SUM(declined)`, como el valor `False` es 1, si la suma es 3, todas fueran rechazadas.

Para la condición `Activa`, si alguna ha sido aceptada, utilicé el `MIN(declined)` para saber si algún valor es 0 (`True`). Si la condición es verdadera, entonces la tarjeta está activa.

Otra posibilidad es `WHEN..ELSE`. Si la primera condición `SUM(declined)` no es igual a 3 (`Activa`), pues `ELSE (Falsa).`

Exericici 1

Quantes targetes estan actives?

```
-- Exercici 1
-- Quantes targetes estan actives?
SELECT COUNT(tarjeta) AS Tarjetas_activas
FROM situacion_tarjetas
WHERE Activa_Inactiva = 'Activa';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Tarjetas_activas
4995

Result 2

Output

Action Output

#	Time	Action	Message
1	11:04:35	SELECT COUNT(tarjeta) AS Tarjetas_activas FROM situacion_tarjetas WHERE Activa_In...	1 row(s) returned

_ - _

Nivell 3

Crea una taula amb la qual puguem unir les dades del nou arxiu products.csv amb la base de dades creada, tenint en compte que des de transaction tens product_ids. Genera la següent consulta:

Esa es la estrella del sprint. La tabla puente. El problema reside en el campo `product_ids` de la tabla `transaction`, que tiene 3 datos por celda, lo que viola la Convención de Ginebra de las tablas relacionadas, si el campo es llave, es UN DATO POR CAMPO. Además, también en *mala praxis* en general, cada dato por campo no es burocracia del modelo relacional, es lo que permite que las consultas sean rápidas, limpias y mantenibles. Cómo `product_ids` guarda varios IDs en una sola celda, no podemos vincularlo directamente con un JOIN normal. Además, tenemos que cambiar el tipo de dato de `product_id` a `INT` para crear las llaves y si hay comas, el campo por fuerza es `string (VARCHAR)`.

`JSON_TABLE` nos permite descomponer esos valores en filas individuales, creando así la tabla puente que el modelo relacional exige — una fila por relación, una sola clave por campo.

Pasamos de eso:

transaction_id	product_id
CcS-6894	12, 20, 36

A eso:

transaction_id	product_id
CcS-6894	12
CcS-6894	20
CcS-6894	36

Pasos a seguir:

1º - Creamos la query de JSON_TABLE para devolver los valores como queremos.

2º - Creamos la tabla

3º - Poblemos la tabla con la query del paso 1º.

Paso 1: crear la query JSON_TABLE

```
SELECT
    t.id AS transaction_id,
    jt.product_id
FROM transaction t
CROSS JOIN JSON_TABLE(
    CONCAT('[', REPLACE(t.product_id, ' ', ''), ']'),
    '$[*]' COLUMNS (
        product_id INT PATH '$'
    )
) AS jt;
```

JSON_TABLE no es una tabla real, es una función que genera filas on the go. El CROSS JOIN une cada fila de `transaction` con las filas que genere esa función. Por eso no puede ser `LEFT`, `RIGHT` o `INNER`, porque no hay un `ON`. Es decir, sobre cada fila que iteramos, vamos guardando transaction_id y product_id. Queremos todo.

Desglosamos la parte más fea del código:

```
CONCAT('[', REPLACE(t.product_id, ' ', ''), ']')
```

`REPLACE` convierte "12, 20, 36" en "12,20,36" (elimina espacios)

`CONCAT` convierte "12,20,36" en "[12,20,36]" (JSON válido)

```
'$[*]' COLUMNS (
```

`$` → representa el elemento raíz, en este caso el array entero [12,20,36]

`[*]` → significa "cada elemento del array"

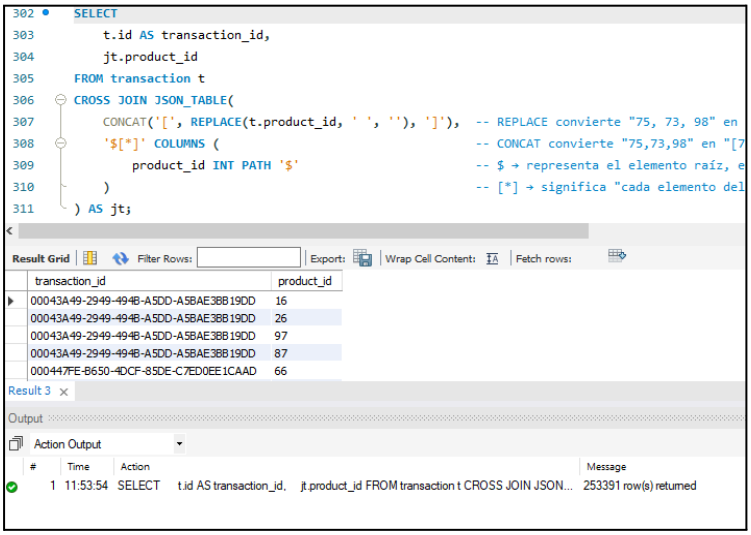
```
product_id INT PATH '$'
```

`product_id` → nombre de la columna nueva

`INT` → tipo de dato

`PATH '$'` → el valor de cada elemento ($ aquí representa cada número individualmente: 12, luego 20, luego 36). Sintaxis de JSON_TABLE.

Ejecutamos esa query:



253391 rows returned.

Paso 2: creamos la tabla `transaction_product` con sus relaciones, tanto primarias como foráneas:

```
CREATE TABLE transaction_product (
    transaction_id CHAR(36) NOT NULL,
    product_id INT NOT NULL,
    PRIMARY KEY (transaction_id, product_id),
    FOREIGN KEY (transaction_id) REFERENCES transaction(id),
    FOREIGN KEY (product_id) REFERENCES product(id)
);
```

Los campos son primary keys para evitar duplicados que se puedan crear. Como visto, ahora `transaction_product.product_id` es `INT`. Podemos relacionarla con `product.id`.

Paso 3: poblamos la tabla recién creada con los valores de la query del paso 1:

```
INSERT INTO transaction_product (transaction_id, product_id)
SELECT
    t.id AS transaction_id,
    jt.product_id
FROM transaction t
CROSS JOIN JSON_TABLE(
    CONCAT('[', REPLACE(t.product_id, ' ', ''), ']'),
    '$[*]' COLUMNS (
        product_id INT PATH '$'
    )
) AS jt;
```

```
1 •  SELECT * FROM global_transactions.transaction_product;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch

transaction_id	product_id
00043A49-2949-494B-A5DD-A5BAE3BB19DD	16
00043A49-2949-494B-A5DD-A5BAE3BB19DD	26
00043A49-2949-494B-A5DD-A5BAE3BB19DD	87
00043A49-2949-494B-A5DD-A5BAE3BB19DD	97
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	66

transaction_product 2

Output

Action Output

#	Time	Action	Message
1	12:01:35	SELECT * FROM global_transactions.transaction_product	253391 row(s) returned

Exericici 1

Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

```
SELECT product_id AS Id,
       (SELECT product_name
        FROM product
        WHERE product.id = transaction_product.product_id) AS Producto,
       COUNT(transaction_id) AS Venta
FROM transaction_product
WHERE transaction_id = (SELECT id
                        FROM transaction
                        WHERE transaction.id = transaction_product.transaction_id
                        AND declined = 0)
GROUP BY product_id
ORDER BY Venta DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Id	Producto	Venta
52	riverlands the duel	2642
29	Tully maester Tarly	2627
21	duel Direwolf	2603
16	the duel warden	2602
33	duel warden	2593
87	sith Jade	2591

Result 4

Output

Action Output

#	Time	Action	Message
1	12:04:15	SELECT product_id AS Id, (SELECT product_name FROM product WHERE pr...	100 row(s) returned

Aquí hay una subconsulta en el `SELECT` para mostrar el nombre del producto y no solamente ID y Venta (lo que te había comentado, dato con contexto). Sé que el ID 52 es el más vendido, pero ¿Qué producto es?

Recomendación de Erik de meter el `declined = 0` para filtrar los que se vendieron de verdad, sin incluir los que la transacción fue rechazada.

Y colorín colorao, este sprint se ha acabao.